

บทที่ 2

ภาษาไพธอน (Python)

ภาษาไพธอนคือภาษาสคริปต์ซึ่งมีการทำงานแบบอินเทอร์พรีเตอร์, เป็นภาษาระดับสูง (High-level language), มีความสามารถเชิงวัตถุ (Object-oriented) และมีความสามารถในการเป็นภาษาสคริปต์ฝั่งเซิร์ฟเวอร์ (Server-side scripting language) มีการออกแบบให้ไม่ขึ้นอยู่กับระบบปฏิบัติการใดๆ จึงเหมาะกับการใช้งานเป็นภาษาสคริปต์ ภาษาไพธอนมีซินแทกที่ไม่สวยงามหรูหรา (Rich syntax) แต่เข้าใจได้ง่าย ซึ่งมีประโยชน์กว่าภาษาที่มีซินแทกที่สวยงามหรูหรา เหมือนดังที่ Guido van Rossum ผู้ให้กำเนิดภาษาไพธอนได้กล่าวไว้ว่า “ภาษาที่สวยงามหรูหราเป็นภาระมากกว่าที่จะช่วย” ภาษาไพธอนมีการพัฒนาอย่างต่อเนื่องโดยไพธอนออแกนไนเซชัน (Python Organization) เวอร์ชันล่าสุดคือ 2.2.1 โดยมีผู้พัฒนาหลายแห่งให้การสนับสนุน ภาษาไพธอนเป็นภาษาที่มีการเปิดเผยซอร์สโค้ด (Open source language) และสามารถดาวน์โหลดตัวแปลภาษาได้จาก www.python.org โดยไม่เสียค่าใช้จ่ายใดๆ

ภาษาไพธอนมีความยืดหยุ่นสูง สามารถนำมาใช้งานได้หลากหลาย และนำกลับมาใช้ใหม่ได้ ในโครงการนี้ได้เลือกทำอินเทอร์พรีเตอร์ขนาดเล็กสำหรับภาษาไพธอน เนื่องจากความสามารถดังที่ได้กล่าวมาแล้วทำให้มีความง่ายและสะดวกเป็นอย่างมากในการพัฒนาตัวภาษาเพื่อใช้กับบราวเซอร์ได้

2.1 คุณสมบัติของภาษาไพธอน

ภาษาไพธอนมีคุณสมบัติ และมีความสามารถหลายประการดังนี้

- **High-level**

ภาษาไพธอนมีโครงสร้างข้อมูลระดับสูงซึ่งช่วยลดระยะเวลาในการพัฒนาโปรแกรมลง เช่น Python's list (Resizable arrays) และ Dictionary (Hash table) ซึ่งการสร้างสิ่งเหล่านี้ในภาษาอื่นๆ เช่น ในภาษา C ทำได้ยากเนื่องจากความจำเป็นที่จะต้องใช้โครงสร้างข้อมูล และ pointer ในภาษา C ข้อดีของการมีโครงสร้างข้อมูลระดับสูงคือ ลดระยะเวลาในการพัฒนาโปรแกรม, ขนาดของโปรแกรม และทำให้โปรแกรมอ่านง่ายซึ่งจะส่งผลให้การ debug โปรแกรมทำได้โดยง่าย

- **Object-oriented**

ภาษาไพธอนถูกสร้างมาเป็นภาษาเชิงวัตถุ (Object Oriented Language) เช่นเดียวกับ C++ และ JAVA และสามารถสร้างคลาสย่อยจากภาษาทั้งสองได้อีกด้วย จึงเหมาะแก่การเป็นภาษาสคริปต์ให้กับภาษาทั้งสอง

- **Scalable**

ภาษาไพธอนมีการเขียนซอร์สโค้ดที่เข้าใจง่าย มีโครงสร้างระดับสูง และสามารถสร้างแพ็คเกจ (package) ที่บรรจุหลายๆคอมโพเน้น (component) ซึ่งสิ่งเหล่านี้สนับสนุนการออกแบบโปรแกรมเป็น

บล็อกของโปรแกรม และเมื่อต้องการจะขยายโปรแกรมโครงสร้างที่เป็นโมดูล และ pluggable ของไพธอนทำให้สามารถจัดการขยายได้โดยง่าย

- **Extensible**

การเขียนโปรแกรมในภาษาไพธอนสามารถแบ่งซอร์สโค้ดออกเป็นโมดูลย่อยซึ่งโมดูลย่อยเหล่านี้และโมดูลใน standard library ของไพธอน ผู้เขียนโปรแกรมสามารถเรียกมาใช้ได้ นอกจากนี้ยังสามารถห่อ(wrapping) lower-level code โดยใช้ไพธอน interface ซึ่งทำให้สามารถสร้างและ import โมดูลที่คอมไพล์แล้วเข้ามาใช้งานได้ โดยวิธีการเรียกใช้งานยังคงเหมือนโมดูลปกติในไพธอนแต่ประสิทธิภาพในการทำงานจะดีขึ้น ซึ่งความสามารถนี้ทำให้ไพธอนสามารถ extend โมดูลจากภาษาอื่นได้ด้วย เช่น ภาษาJava สำหรับ Jpython

- **Portable**

ภาษาไพธอนถูกพัฒนาขึ้นมาจากภาษา C โปรแกรมที่เขียนจากภาษาไพธอนจะสามารถใช้ได้กับระบบปฏิบัติการทั่วไป โดยตัวแปลภาษาจะแปลคำสั่งให้เป็น bytecode ที่ระบบปฏิบัติการนั้นเข้าใจ

- **Easy-to-learn**

ภาษาไพธอนมีคีย์เวิร์ดน้อย มีโครงสร้างภาษาที่เรียบง่าย และมี syntax ที่เข้าใจง่ายทำให้สามารถเรียนรู้ได้ง่าย และรวดเร็ว

- **Easy-to-read**

syntax ของภาษาไพธอนไม่มีการใช้สัญลักษณ์ที่มักพบในภาษาอื่นๆ ที่ใช้สำหรับในการเข้าถึงตัวแปร, การกำหนดสโคปของ Code block และการทำ pattern matching เช่น dollar sign(\$), semicolons(;), tildes(~) ซึ่งการที่ภาษาไพธอนหลีกเลี่ยงการกระทำดังกล่าวทำให้ซอร์สโค้ดของภาษาไพธอนสามารถอ่านทำความเข้าใจได้ง่าย

- **Easy-to-maintain**

เนื่องจากภาษาไพธอนมีคุณสมบัติ easy-to-learn และ easy-to-read ทำให้เกิดข้อดีคือ มีความซับซ้อนน้อยลง จึงทำให้การบำรุงรักษา และการพัฒนาต่อทำได้โดยง่าย

- **Robust**

ภาษาไพธอนมีการจัดการข้อผิดพลาดที่ดี โดยเมื่อไพธอนเกิดมีการทำงานที่ผิดพลาดขึ้นก็จะมี stack trace ซึ่งจะระบุว่าเกิดข้อผิดพลาดขึ้นที่ไหน และอย่างไรบ้าง และภาษาไพธอนยังมีความสามารถในการให้ผู้เขียนโปรแกรมเพื่อจัดการกับข้อผิดพลาด(Exception handler) อีกด้วย

- **Effective as a Rapid Prototyping Tool**

การเขียนโปรแกรมบนระบบใดๆโดยใช้ภาษาไพธอนเพียงอย่างเดียวสามารถทำได้ ถึงแม้ว่า compiled language อื่นๆ ก็สามารถทำได้เช่นกัน แต่ไพธอนมี library จำนวนมากให้ใช้ทำให้ลดระยะเวลาในการพัฒนาได้ ซึ่ง library ที่มีมาในตัวแปลภาษาอยู่แล้ว เช่น networking, Internet/Web/CGI, graphics and graphical user interface (GUI) development(Tkinter), imaging(PIL), numerical computation and analysis(Numpy), database access, hypertext(HTML/XML/SGML)

- **A Memory Manager**

ข้อเสียที่สำคัญของการเขียนโปรแกรมในภาษา C และ C++ คือ การที่ผู้เขียนโปรแกรมต้องรับผิดชอบในการจัดการกับหน่วยความจำ แต่ในภาษาไพธอนตัวแปลภาษาจะทำหน้าที่นี้ให้โดยอัตโนมัติ ทำให้ผู้เขียนโปรแกรมไม่ต้องมากังวลกับการจัดการหน่วยความจำ ซึ่งส่งผลดีคือ ทำให้โปรแกรมมีข้อผิดพลาดน้อยลง และระยะเวลาในการพัฒนาน้อยลง

- **Interpreted and (Byte-) Compiled**

ไพธอนเป็นภาษาที่มีการทำงานแบบInterpreter ทำให้การทำงานช้ากว่าภาษาที่ใช้การ compile เนื่องจากการประมวลผลคำสั่งไม่ได้ทำในรูปแบบของภาษาเครื่องของระบบ(system's native binary language) แต่ภาษาไพธอนก็สามารถทำ byte-compiled ได้ซึ่งผลจากการ compile จะอยู่ในรูปแบบที่ใกล้เคียงกับภาษาเครื่องทำให้ประสิทธิภาพการทำงานดีขึ้น และยังมีความเป็นภาษาที่มีการทำงานแบบ Interpreter อยู่

	Execution Speed	Coding Speed	Object-Oriented	GUI Coding	Dev Environment	Suitability for large tasks	Libraries available
Python	Fair	Excellent	Excellent	Good	Fair	Excellent	Good
Perl	Fair	Excellent	Fair	Good	Fair	Fair	Excellent
Vis.Basic	Good	Excellent	Fair	Excellent	Excellent	Poor	Fair
C	Excellent	Poor	Poor	n/a	Excellent	Good	Good
C++	Excellent	Fair	Excellent	n/a	Excellent	Excellent	Good
Java	Fair	Good	Excellent	Good	Excellent	Excellent	Good

ตารางที่ 2-1 ตารางเปรียบเทียบคุณสมบัติต่างๆ ของภาษาไพธอนกับภาษาอื่นๆ

2.2 ชนิดของข้อมูล

ชนิดของข้อมูลในภาษาไพธอนจะอยู่ในอ็อบเจกต์ (object) ที่ชื่อว่า ไพธอนอ็อบเจกต์ การกำหนดชนิดของข้อมูลให้กับตัวแปรในภาษาไพธอนจะถูกกระทำโดยอัตโนมัติ ผู้เขียนโปรแกรมสามารถเปลี่ยนแปลงตัวแปรหนึ่งให้เก็บค่าที่ต่างไปได้โดยไม่ต้องสนใจชนิดของตัวแปรที่เก็บอยู่เดิม ชนิดของข้อมูลทั่วไปได้แก่

2.2.1 ตัวเลข (Numeric)

ข้อมูลชนิดตัวเลข ได้แก่

2.2.1.1 เลขฐานสิบ (Integer) คือเลขที่ประกอบไปด้วยเลข 0-9 ไม่ว่าจะเป็นจำนวนบวก หรือจำนวนลบแต่ต้องไม่มีจุดทศนิยม ตัวอย่างเช่น 1, -3, 8784 ฯลฯ

2.2.1.2 เลขฐานแปดคือ ตัวเลขที่ขึ้นต้นด้วย 0 เช่น 0715 ฯลฯ

2.2.1.3 เลขฐานสิบหกคือ ตัวเลขที่ขึ้นต้นด้วย 0x เช่น 0x9FAC, 0x12A ฯลฯ

2.2.1.4 เลขทศนิยม(Floats) เช่น 3.2, -45e12 ฯลฯ

2.2.1.5 เลขจำนวนเต็มขนาดยาว(Long integer) คือตัวเลขที่ลงท้ายด้วยตัว L หรือ l เช่น 4890549579L, 5l ฯลฯ

2.2.1.6 เลขจำนวนเชิงซ้อน(Complex number) เช่น 9j, 5i-4j, 45i ฯลฯ

2.2.2 จำนวนทางตรรกะ (Boolean)

โดยปกติจำนวนทางตรรกะจะมีค่าเท็จ หรือจริง สำหรับภาษาไพธอนจะแทนค่าเท็จด้วย เลขศูนย์, โครงสร้างเปล่า หรือ None value เช่น 0, [], {}, (), None และแทนค่าจริงด้วย ค่าใดๆที่ไม่ใช่ศูนย์, โครงสร้างที่มีข้อมูลอยู่ เช่น 1, [5], (7,8,98,347), "xyz"

2.2.3 สายอักขระ (String)

ข้อมูลชนิดสายอักขระ คือ ข้อมูลชนิดของตัวอักษร เช่น 'This is a string.', "This is another string" สายอักขระนี้จะอยู่ภายในสัญลักษณ์ Single quoted หรือ Double quoted ก็ได้ แต่ต้องเหมือนกันทั้งหัว และท้าย หากใช้ Single quoted ในสายอักขระจะสามารถมี Double quoted ได้ และในทำนองเดียวกัน หากใช้ Double quoted ก็จะสามารถมี Single quoted ในสายอักขระได้เช่นเดียวกัน

ในข้อมูลชนิดสายอักขระนั้นจะมีอักขระพิเศษอยู่ด้วย ได้แก่

\n = Newline \' = Single quoted \b = Backspace

\t = Tab \" = Double quoted \f = Formfeed

\\ = Backslash \a = Bell \r = Carriage return

\v = Vertical tab

สำหรับการใช้ ‘’ หรือ “” นั้นจะเป็นสายอักขระที่ หากมีการขึ้นบรรทัดใหม่ในสายอักขระ หรือ มีการย่อหน้าจะทำการเปลี่ยนให้เป็นตัวอักขระพิเศษ

2.2.4 ลิสต์ (Lists)

ข้อมูลชนิดลิสต์นี้จะเป็นการนำเอาข้อมูลชนิดอื่นๆ มาเก็บไว้โดยข้อมูลชนิดนี้ โดยจะมีลักษณะคล้ายกับข้อมูลชนิดอาร์เรย์ (Array) แต่จะต่างกันตรงที่ข้อมูลที่เก็บในลิสต์ไม่จำเป็นต้องเก็บข้อมูลชนิดเดียวกัน ข้อมูลที่สามารถเก็บในลิสต์ได้แก่ ตัวเลข, ตัวอักษร, สายอักขระ หรือแม้แต่จะเป็นลิสต์ด้วยตัวเองก็ได้

การเขียนอ้างอิงถึงข้อมูลชนิดลิสต์ทำได้ดังนี้

<code>list1 = []</code>	เป็นการสร้างลิสต์เปล่า
<code>list2 = [1, "two", [3, 4]]</code>	เป็นการสร้างลิสต์ที่ประกอบด้วยสมาชิก 3 ตัว โดยตัวแรกเป็น ตัวเลข, ตัวที่สองเป็นสายอักขระ, ตัวที่สามเป็นลิสต์ที่มีสมาชิกเป็นตัวเลขสองตัว

การอ้างอิงถึงสมาชิกในลิสต์ทำได้ดังนี้

<code>list2[0]</code>	จะได้ผลลัพธ์เป็น 1
<code>list2[1:2]</code>	จะได้ผลลัพธ์เป็น "two"
<code>list2[1:]</code>	จะได้ผลลัพธ์เป็น ["two", [3, 4]]
<code>list2[-1]</code>	จะได้ผลลัพธ์เป็น [3, 4]

2.2.5 ทัปเปิล (Tuples)

ข้อมูลชนิดทัปเปิลมีลักษณะเหมือนข้อมูลชนิดลิสต์ แต่จะต่างกันตรงที่ไม่สามารถแก้ไขข้อมูลของสมาชิกภายในทัปเปิลได้ ทัปเปิลจึงเหมาะแก่การใช้เป็นข้อมูลอ้างอิง การกำหนดทัปเปิลมีวิธีการดังนี้

<code>tuple1 = ()</code>	เป็นการสร้างทัปเปิลเปล่า
<code>tuple2 = ("one", 2, "three", 4)</code>	

การจัดการกับทัปเปิลนั้นมีวิธีการเช่นเดียวกับการจัดการกับลิสต์

2.2.6 ดิกชันนารี (Dictionary)

ข้อมูลชนิดดิกชันนารีจะประกอบไปด้วยคีย์ (Keys) และค่าที่เก็บ (Values) ในการอ้างอิงข้อมูลประเภทดิกชันนารีทำได้ดังนี้

<code>dic[key]</code>	จะให้ผลลัพธ์เป็นอ็อบเจกต์ที่ถูกเก็บโดยคีย์นั้น
<code>dic.has_key(key)</code>	จะส่งค่ากลับมาเป็น 1 เมื่อมีคีย์ที่ระบุอยู่ใน dic และ 0 เมื่อไม่มีคีย์ที่ระบุอยู่ใน dic
<code>dic.keys()</code>	ส่งค่ากลับมาเป็นลิสต์ของคีย์ต่างๆ ที่มีอยู่ใน dic
<code>dic.values()</code>	ส่งค่ากลับมาเป็นลิสต์ของค่าที่เก็บต่างๆ ที่มีอยู่ใน dic
<code>dic.clear()</code>	ทำการลบทุกค่าที่มีอยู่ใน dic

โดยปกติข้อมูลชนิดลิสต์จะสามารถเรียงลำดับข้อมูลได้ แต่ข้อมูลชนิดคิกซ์นารีนันจะไม่มีการเรียงลำดับข้อมูล การอ้างอิงทำได้โดยผ่านทางคีย์เท่านั้น

2.2.7 ข้อมูลเปล่า (None)

ข้อมูลชนิดนี้ใช้ในการระบุค่าเริ่มต้นของตัวแปร หรือแสดงว่าไม่มีข้อมูล ซึ่งในการเปรียบเทียบค่า None จะมีค่าเท่ากับ None เท่านั้น

2.3 ตัวกระทำ (Operators)

ตัวกระทำต่างๆแบ่งออกเป็น 4 ประเภทคือ ตัวกระทำทางตรรกะ, ตัวกระทำทางการเปรียบเทียบ, ตัวกระทำทางบิตไวส์ และตัวกระทำทางคณิตศาสตร์

2.3.1 ตัวกระทำทางตรรกะ (Logical operators)

ตัวกระทำทางตรรกะมีทั้งหมด 3 ตัวด้วยกันคือ and, or, not โดยทั้ง 3 ตัวกระทำนี้สามารถใช้งานได้ด้วยการเขียน and,or และ not โดยตรง

2.3.2 ตัวกระทำทางการเปรียบเทียบ (Comparison operators)

ตัวกระทำที่ใช้เปรียบเทียบค่าต่างๆมีดังต่อไปนี้

<	น้อยกว่า	>	มากกว่า
<=	น้อยกว่าหรือ เท่ากับ	>=	มากกว่าหรือ เท่ากับ
=	เท่ากับ	<>,!=	ไม่เท่ากับ(สามารถใช้ได้ทั้ง 2 แบบ)
in	x in y หมายความว่าค่า x เป็นสมาชิกใน y หรือไม่		
not in	ให้ผลตรงข้ามกับ in		
is	x is y หมายความว่า x เป็นสิ่งเดียวกับ y ซึ่งไม่เหมือนกับ ==		
is not	ให้ผลตรงข้ามกับ is		

2.3.3 ตัวกระทำทางบิตไวส์ (Bitwise operators)

<<	integer<<n	ส่งค่ากลับเป็นตัวเลขที่เกิดจากการเลื่อนบิตของ integer ไปทางซ้าย (Shift left)ไปเป็นจำนวน n ตัว
>>		ทำงานเช่นเดียวกับ << แต่เลื่อนไปทางขวา(Shift right)
&	m&n	ผลลัพธ์คือการ and กันในรูปตัวเลขฐานสองของ m และ n
	m n	ผลลัพธ์คือการ or กันในรูปตัวเลขฐานสองของ m และ n
^	m^n	ผลลัพธ์คือการ xor กันในรูปตัวเลขฐานสองของ m และ n
~	~m	ผลลัพธ์คือการ not ในรูปตัวเลขฐานสองของ m

2.3.4 ตัวกระทำทางคณิตศาสตร์ (Arithmetic-Style operators)

ตัวกระทำทางคณิตศาสตร์นี้สามารถใช้งานไม่เพียงกับตัวเลขเท่านั้น แต่ยังสามารถใช้งานกับข้อมูลชนิดอื่นๆ เช่นสายอักขระได้ ตัวกระทำดังกล่าวได้แก่

- + หากใช้กับตัวเลขจะเป็นการบวกกัน แต่หากใช้กับสายอักขระจะเป็นการนำมาเรียงต่อกัน และหากเป็นลิสต์ หรือทUPLEจะเป็นการรวมสมาชิกของสองลิสต์เข้าเป็นลิสต์เดียว หรือสมาชิกของสองทUPLEเป็นทUPLEเดียว
- ใช้กับตัวเลขเพื่อทำการลบกัน
- * หากใช้กับตัวเลขจะเป็นการคูณกัน แต่หากใช้กับสายอักขระ, ลิสต์ หรือทUPLEจะเป็นการเพิ่มจำนวนของข้อมูลนั้น เช่น 'M'*5 จะได้ 'MMMMM' เป็นต้น
- / ใช้กับตัวเลขเพื่อทำการหาร โดยหากใช้กับจำนวนเต็มผลที่ได้ก็จะเป็นจำนวนเต็ม เศษที่ได้จะถูกปัดลง ตัวกระทำนี้เหมือนกับ div ในภาษา Pascal
- ** ใช้กับตัวเลขเท่านั้นเป็นการยกกำลัง
- % หากใช้กับตัวเลขจะเป็นการหาเศษจากการหาร เหมือนการใช้คำสั่ง mod ในภาษา Pascal หากใช้กับสายอักขระ จะเป็นการบอกรูปแบบในการพิมพ์

2.3.5 ลำดับของตัวกระทำ (Precedence)

ลำดับของตัวกระทำมีลำดับดังนี้

```

or
and
not
<, <=, ==, >=, >, !=, <>, is, in, not, not in
|
^
&
<<, >>
+, -
*, /, %
**
unary+, unary-, unary~

```

2.4 คำสงวน (Reserved words)

คำสงวนในภาษาไพธอนแบ่งเป็น คีย์เวิร์ด(Keywords) และบิวท์อินฟังก์ชัน(Built-in function) คีย์เวิร์ด ได้แก่

'and', 'assert', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'exec', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'not', 'or', 'pass', 'print', 'raise', 'return', 'try', 'while'

บิวท์อินฟังก์ชันได้แก่

'__import__', 'abs', 'apply', 'buffer', 'callable', 'chr', 'cmp', 'coerce', 'compile', 'complex', 'delattr', 'dir', 'divmod', 'eval', 'execfile', 'filter', 'float', 'getattr', 'globals', 'hasattr', 'hash', 'hex', 'id', 'input', 'int', 'intern', 'isins', 'issubclass', 'len', 'list', 'locals', 'long', 'map', 'max', 'min', 'oct', 'open', 'ord', 'pow', 'range', 'raw_input', 'reduce', 'reload', 'repr', 'round', 'setattr', 'slice', 'str', 'tuple', 'type', 'vars', 'xrange'

2.5 กลุ่มของโค้ด (Code block)

การกำหนดสโคปของซอร์สโค้ดใช้การย่อหน้า(Indentation) แทนสัญลักษณ์ ซึ่งต่างจากภาษาอื่นๆ เช่น ภาษาC และ Java ใช้วงเล็บปีกกา และภาษา Pascal ใช้ begin end เป็นตัวกำหนดสโคปของซอร์สโค้ด

การที่ไพธอนใช้การย่อหน้าในการกำหนดสโคปของซอร์สโค้ดทำให้ช่วยลดความยุ่งยาก และข้อผิดพลาดที่มักเกิดขึ้นจากการลืมพิมพ์สัญลักษณ์ที่ใช้ในการกำหนดสโคป และข้อดีอีกอย่างหนึ่งคือ ทำให้ซอร์สโค้ดอ่านง่ายเนื่องจากทุกครั้งที่ขึ้นสโคปใหม่จะต้องทำการย่อหน้าซึ่งทำให้ซอร์สโค้ดมีความเป็นระเบียบ

2.6 คอมเมนต์ (Comment)

สำหรับการคอมเมนต์ นั้นจะใช้ '#' เป็นตัวระบุ โดยข้อความที่อยู่หลัง '#' ไปจนสุดบรรทัดจะไม่ถูกประมวลผลโดยตัวแปลภาษาไพธอน

2.7 การพิมพ์ statement เดียวในหลายบรรทัด (Continuation)

เนื่องจาก statement ในภาษาไพธอนจะถูกแบ่งโดยการขึ้นบรรทัดใหม่(NEWLINE) ดังนั้นหนึ่ง statement จะต้องใช้หนึ่งบรรทัด แต่ว่าสามารถใช้เครื่องหมาย backslash(\) ในการพิมพ์ statement เดียวในหลายบรรทัดได้ดังนี้

```
if (weather_is_hot == 1) and \
    (available == 1):
    goto_the_beach()
```

2.8 การพิมพ์หลาย statement ในบรรทัดเดียว

ใช้เครื่องหมาย semicolon(;) ในการพิมพ์หลาย statement ในบรรทัดเดียว เช่น

```
import sys; x = 'hello'; sys.stdout.write(x+'\n')
```


2.9 การกำหนดค่าให้ตัวแปร (Variable assignment)

การกำหนดค่าให้ตัวแปรใช้เครื่องหมายเท่ากับ (=) ซึ่งการกำหนดค่าให้ตัวแปรนั้นไม่ได้เป็นการนำค่าไปใส่ให้ตัวแปรจริงๆ แต่เป็นการกำหนดให้ตัวแปรอ้างอิงไปยังไพธอนอ็อบเจกต์ที่มีค่านั้นอยู่ โดยไพธอนอ็อบเจกต์นั้นอาจถูกสร้างขึ้นมาใหม่ตอนที่กำหนดค่า หรือเป็นไพธอนอ็อบเจกต์ที่มีอยู่แล้วจากตัวแปรอื่นก็ได้

2.9.1 การกำหนดค่าหนึ่งค่าให้กับหลายตัวแปร (Multiple assignment)

$x = y = z = 1$

ตัวอย่างด้านบนเป็นการกำหนดค่าหนึ่งค่าให้กับหลายตัวแปร โดยอ็อบเจกต์ชนิดจำนวนเต็มที่มีค่า 1 จะถูกสร้างขึ้น และตัวแปร x, y, z จะถูกกำหนดให้อ้างอิงไปยังอ็อบเจกต์นี้ ซึ่งนี่คือขั้นตอนการกำหนดค่าหนึ่งอ็อบเจกต์ให้กับหลายตัวแปร

2.9.2 การกำหนดค่าหลายค่าให้กับหลายตัวแปร

$x, y, z = 1, 2, \text{'three'}$

ตัวอย่างด้านบนเป็นการกำหนดค่าหลายค่าให้กับหลายตัวแปรโดยผลลัพธ์ที่ได้คือ x, y และ z มีค่าเท่ากับ 1, 2 และ 'three' ตามลำดับ การกำหนดค่าแบบนี้ทั้งสองข้างของเครื่องหมายเท่ากับคือทUPLE ซึ่งตัวอย่างด้านบนสามารถเขียนได้อีกอย่างคือ $(x, y, z) = (1, 2, \text{'three'})$

การกำหนดค่าโดยใช้วิธีนี้ยังมีประโยชน์อีกอย่างในกรณีที่ต้องการสลับค่าตัวแปรซึ่งสามารถเขียนเพียง statement เดียวก็สามารถสลับค่าตัวแปรได้แล้ว เช่น

$x, y = y, x$ หรือ $(x, y) = (y, x)$

แต่ถ้าใช้ภาษาอื่น เช่น ภาษา C ต้องเขียนดังนี้

$\text{temp} = x$

$x = y$

$y = \text{temp}$

2.10 คำสั่งควบคุมการทำงาน (Control flow)

คำสั่งรูปแบบคำสั่งที่ใช้ในการควบคุมลำดับการประมวลผลของคำสั่งต่างๆ โดยให้ประมวลผลวนรอบไปเรื่อยๆจนกว่าจะถึงเงื่อนไขที่กำหนด คำสั่งรูปแบบต่างๆได้แก่ IF, WHILE, FOR

2.10.1 คำสั่ง IF

รูปแบบของคำสั่ง IF มีดังนี้

If <EXPRESSION>:

<STATEMENT>

elif <EXPRESSION>:

<STATEMENT>

else:

<STATEMENT>

โดย <EXPRESSION> นั้นหมายถึงเงื่อนไขในการหยุดลูป ส่วน <STATEMENT> คือคำสั่งที่อยู่
ในลูป โดยตัวอย่างของการใช้คำสั่ง IF เช่น

```
if x == 0:
    print 'x equal 0'
elif x > 0:
    print 'x is positive number'
else:
    print 'x is negative number'
```

2.10.2 คำสั่ง WHILE

คำสั่ง WHILE นั้นมีการตรวจสอบเงื่อนไขก่อนการทำคำสั่งในลูปทุกครั้ง โดยมีรูปแบบคือ

```
while <EXPRESSION> :
    <STATEMENT>
```

ตัวอย่างคำสั่ง WHILE เช่น

```
while 1:
    Print 'Infinite loop'
```

2.10.3 คำสั่ง FOR

คำสั่ง FOR เป็นคำสั่งที่จะวนรอบการทำงานจนครบจำนวนที่กำหนด รูปแบบของคำสั่งคือ

```
For <VARIABLE> in <RANGE>:
    <STATEMENT>
```

โดย <RANGE> คือตัวแปรที่เป็นลิสต์ หรือคำสั่งเฉพาะบางคำสั่ง ส่วน <VARIABLE> คือตัวแปรที่เก็บค่าของสมาชิกทุกตัวของ <RANGE> การทำงานของคำสั่ง FOR จะทำงานวนรอบเป็นจำนวนเท่ากับจำนวนสมาชิกของ <RANGE> ตัวอย่างเช่น

```
list1 = [1,'a',[b,c]]
for var1 in list1:
    print var1
```

จากซอร์สโค้ดข้างต้นได้ผลลัพธ์ดังนี้

```
1
'a'
[b,c]
```

การทำงานของคำสั่ง FOR ข้างต้นจะทำงานเป็นจำนวน 3 ครั้ง เท่ากับจำนวนสมาชิกใน list1

```
for i in range(8):
```

```
    print i
```

คำสั่ง range(n) จะส่งค่ากลับเป็นตัวเลขตั้งแต่ 0 จนถึง n-1 ผลลัพธ์ที่ได้คือ

```
0
```

```
1
```

```
2
```

```
3
```

```
4
```

```
5
```

```
6
```

```
7
```

2.11 การจัดการหน่วยความจำ (Memory management)

ลักษณะเด่นของตัวแปร และการจัดการหน่วยความจำที่ผู้ใช้งานสามารถสังเกตได้มีดังนี้

- ตัวแปรจะถูกประกาศตอนให้ค่าครั้งแรก
- ไม่มีการกำหนดชนิดของตัวแปร
- ผู้เขียนโปรแกรมไม่ต้องเขียนโปรแกรมจัดการกับหน่วยความจำ
- ชื่อตัวแปรที่เคยประกาศใช้ไปแล้วสามารถนำมาใช้ใหม่ได้ถึงแม้ว่าจะมีชนิดของตัวแปรที่ต่างกัน
- สามารถใช้คำสั่ง del เพื่อทำการปลดปล่อยหน่วยความจำอย่างเปิดเผยในการเขียนโค้ดได้

2.11.1 การประกาศตัวแปร (Variable Declarations)

ตัวแปรจะถูกประกาศตอนให้ค่าครั้งแรกด้วยโอเปอเรเตอร์เท่ากับ (Assignment operation) โดยไม่ต้องมีการกำหนดชนิดของตัวแปร

2.11.2 การจองหน่วยความจำ (Memory Allocation)

ผู้เขียนโปรแกรมไม่ต้องจัดการเกี่ยวกับการจอง และการปลดปล่อยหน่วยความจำ เนื่องจากไพธอนจะจัดการกับการทำงานในระดับต่ำ (Lower-level) เช่นนี้ให้อยู่แล้ว

2.11.3 การรวบรวมหน่วยความจำที่ไม่ได้ใช้ (Garbage Collection)

หน่วยความจำที่ไม่ได้ใช้ ระบบจะมีกลไกในการรวบรวมหน่วยความจำเหล่านี้กลับมาใช้ใหม่ โดยที่ผู้เขียนโปรแกรมไม่ต้องเขียนโค้ดในส่วนนี้เนื่องจากไพธอนจะใช้การนับการอ้างอิง (Reference counting) ในการตัดสินใจว่าหน่วยความจำส่วนไหนที่ไม่ได้ใช้อีกต่อไปอีกแล้ว

2.11.4 การนับการอ้างอิง (Reference counting)

ไพธอนจะมีตัวแปรที่ใช้สำหรับนับการอ้างอิงที่กระทำไปยังไพธอนอ็อบเจ็กต์ โดยค่าเริ่มต้นของตัวแปรนี้คือ หนึ่ง เมื่อตอนประกาศตัวแปร และเมื่อไพธอนอ็อบเจ็กต์ถูกอ้างอิงอีก หรือถูกใช้เป็นอาร์กิวเมนต์ในการเรียกใช้ฟังก์ชัน หรือเมธอด ค่านี้ก็จะถูกเพิ่มขึ้นอีกหนึ่ง และเมื่อไรที่ค่านี้มีค่าเป็นศูนย์ก็จะหมายถึงไม่มีการอ้างอิงถึงอ็อบเจ็กต์นี้อีกแล้ว และหน่วยความจำในส่วนนี้ก็จะถูกนำกลับมาใช้งานใหม่

```
foo1 = 'foobar'           # reference count = 1
foo2 = foo1               # reference count = 2
Check_value(foo1)         # reference count = 3 ตอนเรียกใช้ และหลังจากใช้เสร็จจะมาเท่าเดิม
```

จากตัวอย่างด้านบนเมื่อประกาศตัวแปรครั้งแรก reference count ของสายอักขระ 'foobar' จะมีค่าเท่ากับหนึ่ง และเมื่อถูกอ้างอิงโดย foo2 ค่า reference count ก็จะถูกเพิ่มเป็นสอง และในตอนที่ใช้ฟังก์ชันก็จะถูกเพิ่มเป็นสาม แต่พอจบจากฟังก์ชันก็จะถูกลดไปเป็นสอง และถ้าใช้คำสั่ง del กับ foo2 ค่า reference count ก็จะถูกลดเป็นหนึ่ง และเมื่อ foo1 หลุดออกจาก scope ค่า reference count ก็จะถูกลดลงเหลือศูนย์ และหน่วยความจำในส่วนนั้นก็จะถูกนำกลับมาใช้งานใหม่

2.11.5 คำสั่ง del

คำสั่ง del มีรูปแบบดังนี้

```
del obj
```

ตัวอย่างเช่น del foo2 จะทำให้เกิดผลสองอย่างดังนี้คือ

- ลบชื่อ foo2 ออกจาก namespace
- ลดค่า reference count ของอ็อบเจ็กต์ที่มันอ้างอิงถึง ('foobar') ลงหนึ่ง

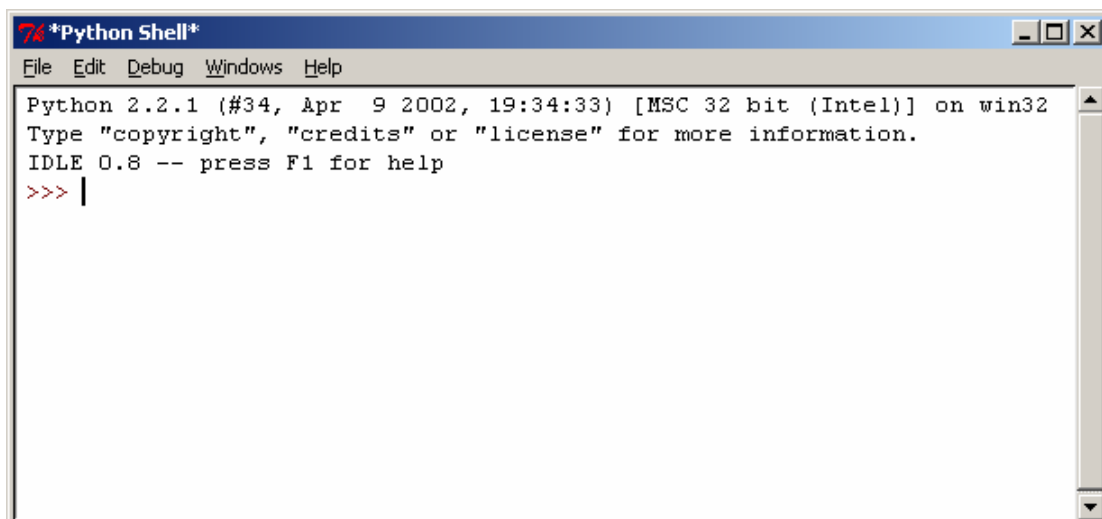
2.11.6 การลดค่า reference count

ค่า reference count จะถูกลดลงเมื่อเกิดเหตุการณ์ต่อไปนี้

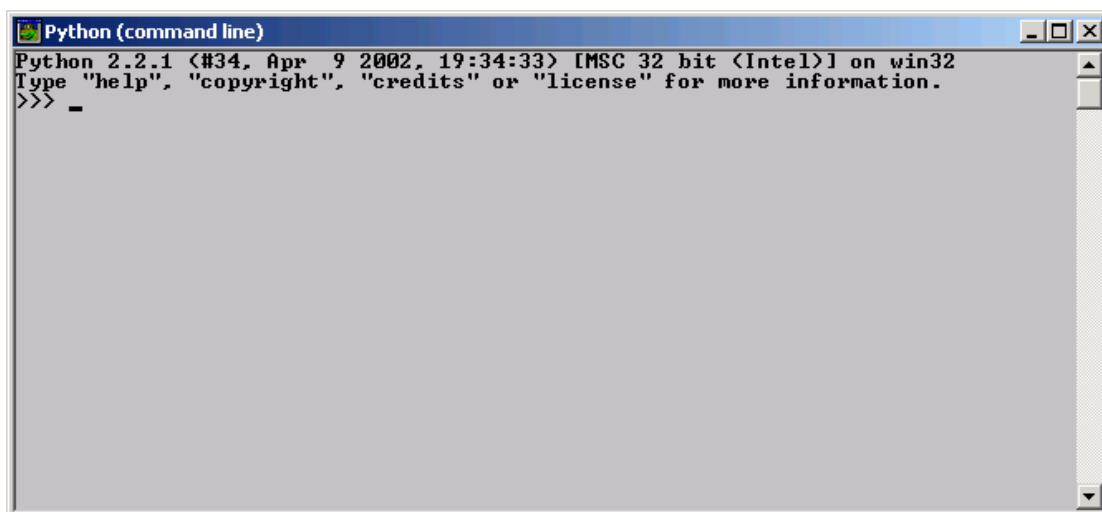
- ใช้คำสั่ง del
- ตัวแปรถูกเปลี่ยนการอ้างอิงจากอ็อบเจ็กต์เดิมไปยังอ็อบเจ็กต์ใหม่ ซึ่งอ็อบเจ็กต์เดิมนั้นก็จะถูกลดค่า reference count ลงหนึ่ง
- หลุดออกจาก scope

2.12 การใช้งานภาษาไพธอน

ภาษาไพธอนที่โครงการเลือกใช้คือภาษาไพธอนเวอร์ชัน 2.2.1 ในชุดของโปรแกรมจะประกอบไปด้วย คู่มือการใช้งานเป็นเอกสาร HTML, และโปรแกรมหน้าจอโต้ตอบซึ่งมีให้เลือกใช้ 2 อย่างคือ IDLE ซึ่งเป็นไพธอน GUI และไพธอนคอมมานด์ไลน์ซึ่งเป็นหน้าจอโต้ตอบอยู่บน DOS



รูปที่ 2-1 แสดงโปรแกรมไพธอน IDLE



รูปที่ 2-2 แสดงโปรแกรมไพธอนคอนโซล